

AFL Analyst

Domain-Scoped AFL Chat & Prediction Assistant — Executive Report

Capstone deliverable · Week 6, Day 5

Product Goal

AFL Analyst is a domain-locked conversational assistant for Australian Rules football: it answers grounded factual questions, retrieves real statistics, and produces probabilistic match/player predictions — while refusing, cleanly and consistently, anything outside AFL. The product goal is to demonstrate a production-ready pattern for a scoped sports-data chatbot suitable for a client property (e.g. Web3Geeks): every number in a response must trace back to a tool call, every prediction carries a clear "not a certainty" disclaimer, and the assistant holds its scope under adversarial pressure.

Architecture

The system is a LangGraph state machine with nine nodes: guard → router → {prediction | retrieval | direct_answer | refusal | ambiguous} → validation → {clarification | fallback | response_formatting}. A layered scope guard runs first on every turn (jailbreak-pattern detection, off-topic keyword weighing, an ambiguity band for AFLW/betting-adjacent questions, and anaphora-aware continuation handling for follow-ups). Prediction calls (match-winner logistic-regression model, top-player ridge-regression model) and retrieval calls (head-to-head, season stats, last-round, ladder) are wrapped in a hard timeout and a try/except boundary, so one failing tool call degrades to a clear message instead of crashing the turn. The graph is exposed through a FastAPI /chat endpoint (message + conversation_id in, response + prediction metadata out) with structured JSON-line logging of every turn, and a lightweight HTML/JS chat UI for live demos.

Note on data: the production feature store (match_features_v2.0.0.parquet, player_features_v2.0.0.parquet, trained joblib artifacts) referenced in the Week 6 Day 2/4 notebooks was not present among the capstone's uploaded files. A schema-compatible synthetic dataset (19 clubs, ~8,900 matches 1983–2025, ~200k player-game rows 1999–2025) was generated to keep the system fully runnable end-to-end; swapping in the real parquet files is a one-file change (app/data.py) with no changes required anywhere else in the pipeline.

Evaluation Results

27 test cases across four categories were run against the live graph (see eval/results_summary.md for full detail):

Category	Passed	Total	Pass rate
Factual	6	7	85.7%
Prediction	7	7	100.0%
Guardrail	8	8	100.0%
Multi-turn	5	5	100.0%
Overall	26	27	96.3%

Weakest category: factual (85.7%).

One case fails honestly rather than being tuned to pass: "How many players are on the field per team?" contains no explicit AFL vocabulary the scope guard recognises, so it is misrouted to off-topic. Concrete

fix: extend the guard to check candidate queries against the internal facts-KB pattern list before falling back to keyword scope-checking, so KB-answerable questions are never lost to a vocabulary gap.

Prompt-injection hold test: 5 adversarial attempts (instruction-override, DAN-style roleplay, "forget everything", system-prompt exfiltration, roleplay reframing) were run — all 5 held AFL-only scope (see eval/injection_tests.py).

Prediction Model vs. Naive Benchmark

The match-winner model was compared against a naive "better recent win-rate wins" heuristic on a held-out set of 911 matches, disjoint from training data:

Predictor	Accuracy (911-match held-out set)
Always pick home team	54.7%
Naive recent-form heuristic	65.6%
Trained model (LogisticRegression)	67.3%

A +1.6 point lift over the naive heuristic is modest but realistic for a lightweight two-feature model — it should be presented to stakeholders as directionally useful, not betting-grade. Real bookmaker-implied accuracy for AFL typically sits in the low-to-mid 70s%, which sets an honest ceiling expectation for "good enough."

Known Limitations

- Data recency: synthetic demo data stands in for the real feature store; production accuracy will differ once real parquet artifacts are loaded.
- Model accuracy ceiling: a two-feature logistic model captures only recent-form signal — no injuries, weather, travel, or head-to-head history are modelled.
- Guardrail edge cases: queries with no explicit AFL vocabulary (see factual F06 above) can be misrouted; continuation/anaphora resolution across topic-switching multi-turn chats is best-effort, not exhaustive.
- Conversation state is in-memory in the FastAPI layer; it will not survive a process restart or scale across multiple workers without a shared store (Redis/DB).

Recommended Next Steps

- Swap app/data.py for the real production parquet feature store and retrain both models on real match/player history.
- Move conversation state to Redis (or equivalent) ahead of any multi-worker deployment.
- Wire the monitoring checklist (see docs/monitoring_checklist.md) into a dashboard (latency, tool-error rate, off-topic leak rate, prediction drift) with the alert thresholds defined there.
- Fix the factual-guard vocabulary gap identified above; add 5–10 more adversarial guardrail cases per release.
- Re-run the naive-vs-model benchmark each time the model is retrained, and track the lift over time as a single headline KPI for stakeholders.